

# Technical case

## Data Engineer

### Business context

**Gross Merchandising Value (GMV)** represents the total monetary value of completed transactions, considering only purchases whose payment has been successfully captured (i.e., *release\_date* is populated) and not canceled.

You are provided with three event-based (CDC) tables (*documented at [link](#)*):

**purchase** (*events/cdc*) - Core transaction event

| purchase (eventos)   |                  |             |          |              |            |              |             |
|----------------------|------------------|-------------|----------|--------------|------------|--------------|-------------|
| transaction_datetime | transaction_date | purchase_id | buyer_id | prod_item_id | order_date | release_date | producer_id |
| 2023-01-20 22:00:00  | 2023-01-20       | 55          | 15947    | 5            | 2023-01-20 | 2023-01-20   | 852852      |
| 2023-01-26 00:01:00  | 2023-01-26       | 56          | 369798   | 746520       | 2023-01-25 | NULL         | 963963      |
| 2023-02-05 10:00:00  | 2023-02-05       | 55          | 160001   | 5            | 2023-01-20 | 2023-01-20   | 852852      |
| 2023-02-26 03:00:00  | 2023-02-26       | 69          | 160001   | 18           | 2023-02-26 | 2023-02-28   | 96967       |
| 2023-07-15 09:00:00  | 2023-07-15       | 55          | 160001   | 5            | 2023-01-20 | 2023-03-01   | 852852      |

**product\_item** (*events/cdc*) - Line items and monetary amounts

| product_item (eventos) |                  |             |            |               |                |
|------------------------|------------------|-------------|------------|---------------|----------------|
| transaction_datetime   | transaction_date | purchase_id | product_id | item_quantity | purchase_value |
| 2023-01-20 22:02:00    | 2023-01-20       | 55          | 696969     | 10            | 50,00          |
| 2023-01-25 23:59:59    | 2023-01-25       | 56          | 808080     | 120           | 2400,00        |
| 2023-02-26 03:00:00    | 2023-02-26       | 69          | 373737     | 2             | 2000,00        |
| 2023-07-12 09:00:00    | 2023-07-12       | 55          | 696969     | 10            | 55,00          |

**Purchase\_extra\_info** (*events/cdc*) - Dimensional attributes such as subsidiary

| transaction_datetime | transaction_date | purchase_id | subsidiary    |
|----------------------|------------------|-------------|---------------|
| 2023-01-23 00:05:00  | 2023-01-23       | 55          | nacional      |
| 2023-01-25 23:59:59  | 2023-01-25       | 56          | internacional |
| 2023-02-28 01:10:00  | 2023-02-28       | 69          | nacional      |
| 2023-03-12 07:00:00  | 2023-03-12       | 69          | internacional |

These tables are ingested asynchronously into the data lake. Events may arrive late, out of order, or be re-sent for historical correction. For every logical purchase, there will always be one corresponding record in each table, but not necessarily on the same ingestion date or time.

### Goal

Your task is to **build an ETL (or ELT) pipeline and a final analytical table** that shows the Daily GMV by subsidiary. The final table should be easy to query by analysts and business users, without requiring complex joins or knowledge of the raw event structure.

## What you need to do

### Understand the data

- Analyze the fields of the three tables.
- Identify how to join them using `purchase_id`.
- Define which records should be included in GMV (only released/paid purchases).

### Design the final table

- Create a table that contains: `transaction_date`, `subsidiary`, `daily_gmv`
- The table should be partitioned by `transaction_date`.

### Build the ETL/ELT Logic

- Join the three tables.
- Filter only valid (paid) purchases.
- Aggregate GMV by day and subsidiary.
- Make the pipeline reproducible (can be rerun without duplicating data).

### Handle data issues

- Deal with possible duplicate events.
- Ensure late-arriving data can be included when the pipeline is reprocessed.
- Keep the table append-only (do not overwrite existing days; reprocess if needed).

### Query the data

- Write a SQL query that returns the daily GMV by subsidiary from your final table.

## Expected Deliverables

### Final table DDL

- Definition of columns and partitions.
- Brief explanation of the table grain (one row per day per subsidiary).

### ETL/ELT script

- In Python, Spark, or SQL.
- Reading the three source tables.
- Joining, filtering, and aggregating into the final table.

### Example output

- A small sample of the final table with mock or real data.

## **SQL query**

- A query that returns daily GMV by subsidiary.

## **Architecture description**

- Short explanation of the tools you used (e.g., Spark, Airflow, dbt, BigQuery, Redshift, etc.).
- How the pipeline would run daily (D-1).

## **What we will evaluate**

- Correct understanding of GMV logic.
- Proper joins and aggregations.
- The data modeling (grain, partitions).
- Clean and readable code.
- Ability to explain your solution clearly.

## **Optional (Nice to have/Bonus)**

- How you would improve data quality checks (nulls, duplicates, unexpected values).
- How you would monitor the pipeline (row counts, GMV deltas, freshness).

# **Sending your solution**

Please, be aware of the deadline and send all your materials (i.e. documentation, codes, files, recording if wanted, etc.) via e-mail.